
Chat-PE: A Natural Language Interface for PyExperimenter

Daniel Kruse¹

¹Leibniz University Hannover

Abstract We present *Chat-PE*, a natural language interface for PyExperimenter that integrates a Large Language Model (LLM) agent with database querying, Python-based analysis, and visualization. The system supports both ReAct and Plan-and-Execute reasoning strategies within a modular architecture.

Evaluation on datasets of increasing complexity shows strong performance on basic aggregation tasks but declining accuracy for complex joins and multi-table multi-step analyses. While the Plan-and-Execute scheme incurs higher computational cost, it does not consistently outperform a basic ReAct agent. The results highlight both the potential and current limitations of LLM-based agents for experiment analytics.

1 Introduction

Modern experimental research in machine learning and data science increasingly relies on structured experiment management systems to ensure reproducibility, scalability, and traceability. Frameworks such as PyExperimenter (Tornede et al. (2023)) enable researchers to define experiment configurations, distribute runs across compute environments, and persist results in relational databases. While such systems provide infrastructure for experiment execution and metadata tracking, interacting with the stored results remains demanding. Researchers typically need to formulate SQL queries, export results into Python, perform analyses manually, and generate visualizations in separate steps.

Recent advances in Large Language Models (LLMs) have demonstrated that natural language can serve as a high-level interface for structured reasoning and tool use. Approaches such as ReAct (Yao et al. (2023)) show that LLMs can effectively combine reasoning traces with tool execution to also handle multi-step problems. Similarly, systems like AskDB (Phan et al. (2025)) utilize such frameworks and illustrate how LLM agents can interact with relational databases. Their results and recent developments suggest that natural language interfaces can reduce friction in analytical workflows while maintaining precision through structured tool execution.

In this project, Chat-PE, an agentic LLM-based conversational interface for experiment databases by PyExperimenter, was developed. Chat-PE allows users to query experiment data, perform analysis through dynamically generated Python code, and generate visualizations directly within a chat interface.

2 Related Works

The development of Chat-PE builds upon several lines of research in natural language interfaces to structured data, tool-augmented language models, and agentic AI systems.

Natural language interfaces to databases have long aimed to bridge the gap between human queries and relational schemas. Recent LLM-based approaches significantly improved performance on complex benchmarks such as Spider (Yu et al. (2019)). AskDB (Phan et al. (2025)) extends this paradigm by introducing an agent that iteratively reasons about schema information and refines SQL queries via tool interaction. However, such systems typically produce single-shot SQL outputs and lack persistent working memory across interactions to tackle multi-step problems.

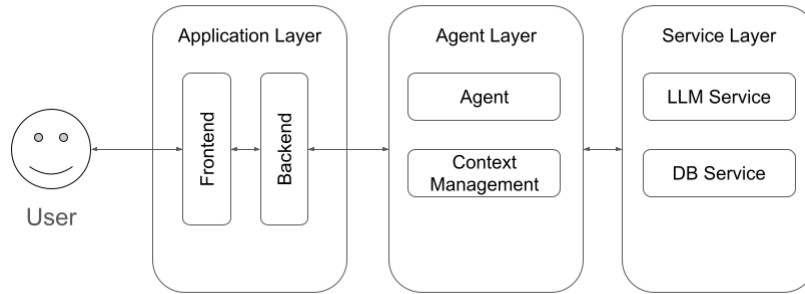


Figure 1: System Architecture of Chat-PE

The ReAct framework (Yao et al. (2023)), that AskDB is based on, demonstrated that combining reasoning traces with tool calls enables LLMs to solve multi-step tasks more reliably. This paradigm inspired a broad class of tool-using LLM systems capable of API calls, database queries, and program execution. Related approaches such as Toolformer (Schick et al. (2023)) and program-aided language models further show that delegating deterministic computation to external interpreters can improve accuracy and robustness.

Beyond single-agent systems, recent work discusses the broader integration of LLM agents into scientific and data-centric workflows. Shin et al. (2025) outline how language models may support planning and execution in research pipelines. Similarly, Fu et al. (2025) frame data-oriented LLM systems as a paradigm for intelligent data interaction. While some approaches explore multi-agent architectures for visualization, such as Wolter et al. (2025) Chat-PE adopts a deliberately simpler single-agent design focused on robust tool integration within experiment management environments.

3 System Architecture

Chat-PE follows a modular Service-Agent-Application architecture (Figure 1) that separates user interaction, reasoning logic, and data access concerns. This layered design enables extensibility and clear separation of responsibilities.

The application layer consists of a Flask-based REST backend and a lightweight single-page frontend implemented in Vanilla JavaScript and CSS. The backend manages user sessions, conversation history, and tool orchestration, while the frontend renders the agent’s responses. Generated figures are displayed directly within the chat interface, with the possibility to save the figures locally. This design provides a responsive, self-contained environment without requiring external notebooks or dashboards.

At the core of the systems lies the agent layer. Two agent implementations are provided. The first follows the ReAct paradigm, decomposing user queries into reasoning steps interleaved with tool invocations. The agent iteratively observes tool outputs and refines its strategy until the task is completed. The second agent follows a Plan-and-Execute strategy, first generating a structured multi-step plan and then executing each step sequentially. The agents have access to three primary tools: a tool to query the experiment database, a Python interpreter in which variables persist across reasoning steps, enabling data cleaning, aggregation, and statistical testing using standard scientific libraries, and a tool to interact with PyExperimenter to reset or restart runs.

The service layer abstracts both data access and LLM communication. It supports two operation modes. In PyExperimenter mode, a configuration file is provided and the system connects via the PyExperimenter library. In Direct Database mode, a SQLite or MySQL connection string is specified, and the system connects directly via SQLAlchemy ensuring access even when original

Dataset	Agent	L1				$\Sigma L1$	L2				$\Sigma L2$	L3				$\Sigma L3$	$\Sigma Total$
		Q1	Q2	Q3	Q4		Q1	Q2	Q3	Q4		Q1	Q2	Q3	Q4		
Synthetic	ReAct	1	1	1	1	4.0	1	1	1	1	4.0	1	0.5	0.5	1	3.0	11.0
Synthetic	PE	1	1	1	1	4.0	0.5	1	1	1	3.5	0.5	1	0.5	1	3.0	10.5
HyperSHAP	ReAct	1	1	1	1	4.0	1	1	1	0	3.0	1	1	0	0.5	2.5	9.5
HyperSHAP	PE	1	1	1	1	4.0	1	1	1	0.5	3.5	1	0	1	0	2.0	9.5
HPI	ReAct	1	1	1	1	4.0	0	0.5	1	0	1.5	0	0	0	0.5	0.5	6.0
HPI	PE	0	1	1	1	3.0	0	0.5	1	0	1.5	0	0	0	0	0.0	4.5

Table 1: Overall evaluation results across datasets and difficulty levels, each consisting of four questions, for the ReAct and Plan-and-Execute Agent.

configuration files are unavailable. All SQL execution is restricted to read-only queries to preserve database integrity. In addition, the service layer manages communication with the LLM backend. By default, it follows the OpenAI API format, but the architecture is designed to be extensible to alternative providers or locally hosted models.

4 Experiments

We evaluate the proposed single-agent system on three datasets of increasing complexity: (i) a small synthetic dataset for controlled initial validation, (ii) an HyperSHAP dataset to assess visualization and reasoning capabilities, and (iii) an HPI dataset to evaluate logtable handling including multi-table joins and trajectory-based analysis. For each dataset, we define three difficulty levels. Level 1 (L1) consists of simple lookups and aggregations, Level 2 (L2) extends these tasks with visualizations or moderate joins, and Level 3 (L3) requires multi-step reasoning, derived metrics, or more complex analytical workflows. Each level contains four questions (Q1–Q4). All of the used questions can be found in the evaluation section in the GitHub repository. We compare our two agent strategies: ReAct and Plan-and-Execute (PE). Especially, Level 3 is interesting for other experiments since it requires the most steps and reasoning capabilities, so it could showcase possible advantages of the Plan-and-Execute scheme that makes a plan before acting. All of the conducted experiments utilize a *"llama-3.3-70b-instruct"* model.

Across all datasets, we observe a consistent reasoning pattern. The agent typically follows a structured sequence of steps: first querying the database, then performing computations in Python, optionally generating a visualization, and finally producing a natural language answer. This “query–compute–(visualize)–answer” pattern appears stable across both agent variants and difficulty levels.

Table 1 summarizes the experiment results. We utilize a simple scoring system to manually grade the agent’s answers. A correct answer gets awarded with a full point, answers with slight inaccuracies or small missing parts receives half a point, and answers missing important information or providing wrong answers get marked with no points. For each level, the sum over the four questions is reported, followed by the total score per dataset.

On the synthetic dataset, both agents achieve near-perfect performance on L1 and L2 tasks, with only minor degradations in L3. On HyperSHAP, performance decreases on L3 tasks that require statistical reasoning and more complex visualization logic. While both agents achieve identical total scores on this dataset, their failure patterns differ slightly.

On the HyperSHAP dataset, which emphasizes visualization and statistical reasoning, performance decreases on L3 tasks for both agents. While both methods achieve identical total scores (9.5), their failure modes differ. ReAct tends to partially complete tasks, whereas PE sometimes fails more decisively on complex reasoning steps.

The HPI dataset introduces significantly higher complexity due to log tables and multi-table joins.

While both agents perform reliably on simple experiment-level queries (L1), performance drops sharply for L2 and L3 tasks. These levels require correct joins between experiment metadata and trajectory logs, aggregation over time, and reasoning about convergence behavior. ReAct achieves a total score of 6.0, whereas Plan-and-Execute reaches 4.5. Notably, neither agent solves more advanced L3 tasks reliably in this setting.

Execution time differences remain consistent across datasets. Plan-and-Execute often requires substantially longer runtimes due to explicit planning phases, but this overhead does not consistently translate into improved accuracy. In the HPI setting, the additional planning does not prevent performance degradation on complex multi-table tasks.

5 Discussion

The results highlight both the strengths and current limitations of LLM-based data agents.

First, structured SQL-based analytics tasks are handled reliably. Basic aggregations and filtering operations (L1) are solved almost perfectly across datasets. This confirms that LLM agents are well-suited for standard experiment database exploration.

Second, both agents demonstrate stable multi-step reasoning behavior. Even for moderately complex tasks, they typically decompose problems into database queries, intermediate computations, and visualization steps.

However, several limitations become apparent, especially in the HPI setting. Multi-table joins and trajectory-based reasoning remain challenging. Schema ambiguity, often not trivially resolvable without additional domain expertise, similar table structures, and the need for temporal aggregation significantly increase error rates. Data type mismatches in Python execution sometimes require multiple correction attempts. Additionally, the model occasionally fails to fully complete tasks, hallucinates successful visualizations or produces figures with poorly readable axis labeling. This may indicate the need for more complex (multi-)agent structures possibly in combination with stronger models.

Comparing the two used strategies reveals a trade-off. ReAct is generally faster and performs competitively across datasets, but can exhibit unstable execution behavior. Plan-and-Execute produces more structured reasoning traces, yet incurs higher computational cost and does not consistently improve accuracy. In more complex settings such as HPI, additional planning alone is insufficient to overcome schema reasoning challenges.

Overall, the experiments suggest that while LLM-based data agents are highly capable for structured experiment analytics, their primary bottlenecks lie in robust schema understanding, reliable multi-table joins, and complex trajectory-based reasoning rather than simple aggregation or visualization tasks.

References

- Fu, Y., Wang, D., Ying, W., Wang, X., Zhang, X., Liu, H., and Pei, J. (2025). Autonomous data agents: A new opportunity for smart data.
- Phan, X.-Q., Mai, T.-H., Dinh, T.-D., Nguyen, M.-T., and Lê, L.-S. (2025). Askdb: An llm agent for natural language interaction with relational databases.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools.
- Shin, W., Souza, R., Rosendo, D., Suter, F., Wang, F., Balaprakash, P., and da Silva, R. F. (2025). The (r)evolution of scientific workflows in the agentic ai era: Towards autonomous science.

- Tornede, T., Tornede, A., Fehring, L., Gehring, L., Graf, H., Hanselle, J., Mohr, F., and Wever, M. (2023). PyExperimenter: Easily distribute experiments and track results. *Journal of Open Source Software*, 8(84):5149.
- Wolter, A., Vidalakis, G., Yu, M., Grover, A., and Dhanoa, V. (2025). Multi-agent data visualization and narrative generation.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). React: Synergizing reasoning and acting in language models.
- Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., and Radev, D. (2019). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task.

AI Assistance

LLMs were utilized in this project to improve the report’s textual clarity and ensuring grammatical consistency. Additionally, AI assisted tools were used to generate parts of the code. While these tools supported the technical and creative process, all final content has been reviewed and verified by the author to ensure accuracy.